

1. Executive Summary

SECURITY SCORE

10

CRITICAL

0

HIGH

10

MEDIUM

11

INFO/LOW

6

1.2 ISMS-P 컴플라이언스 위반 요약

이번 분석에서 총 **34**건의 ISMS-P 인증기준 위반 또는 위반 의심 사례가 탐지되었습니다. 관련 부서는 본 리포트의 세부 항목을 검토 후 조치 요망.

#	ISMS-P 위반 핵심 내용	관련 취약점	심각도
1	ISMS-P 관점에서 안전하지 않은 컨테이너 실행 정책은 접근 통제 및 안전한 시스템 ...	Kubernetes 컨테이너에 권한 상승 방지 설정 누락	MEDIUM
2	ISMS-P 관점에서 최소권한 원칙과 안전한 실행환경 관리에 미흡합니다. 컨테이너가 ...	Docker 컨테이너가 non-root 사용자 없이 실행되어 root 권한으로 동작할 수 있음	HIGH
3	ISMS-P 웹 취약점 분석 항목 16(불충분한 세션 관리) 관점에서 JWT 등 인증...	하드코딩된 JWT 예시 토큰 문자열 사용	HIGH
4	ISMS-P 관점에서 세션 관리 정책(C-14, 불충분한 세션 관리/세션 타임아웃)이...	로그인 폼 조회용 RequestMapping 의 HTTP Method 미지정	MEDIUM

5	ISMS-P 관점에서 최소권한 원칙 및 접근통제 강화 요구에 부합하지 않을 수 있습니다...	Dockerfile에서 실행 사용자(USER) 미지정으로 컨테이너가 root 권한으로 실행될 수 있음	HIGH
6	최소권한 원칙 및 안전한 실행 환경 통제 미흡으로, ISMS-P 관점에서 개인정보 처리...	Kubernetes 컨테이너에 privilege escalation 차단 설정 누락	MEDIUM
7	ISMS-P 관점에서 안전한 서비스 운영을 위한 기술적 보호조치 미흡에 해당할 수 있음...	Kubernetes Pod 컨테이너의 Non-Root 실행 설정 누락	INFO
8	ISMS-P의 최소권한 및 접근통제 취지에 부합하지 않으며, 컨테이너 보안 설정 미흡...	Kubernetes 컨테이너에 권한 상승 방지 설정이 누락됨	MEDIUM
9	명시적 비루트 실행 및 최소권한 통제가 없으면 ISMS-P 보호 대책 요구사항 중 인증...	Kubernetes 컨테이너의 비루트 실행 설정 누락 여부 확인 불가	INFO
10	ISMS-P 관점에서 비인가 권한 사용 제한과 최소권한 원칙이 미흡합니다. 주요정보통...	Kubernetes 컨테이너 비-root 실행 보안 설정 누락	INFO
11	ISMS-P 2.8.1 보안 요구사항 정의 미흡에 해당합니다. 컨테이너 배포 시 최신...	Kubernetes 컨테이너 권한 상승 방지 설정 누락	MEDIUM
12	ISMS-P 기술적 보호조치 측면에서 컨테이너 권한 상승 방지 설정이 미흡하여 최소권...	Frontend 컨테이너의 권한 상승 방지 설정 누락	MEDIUM
13	ISMS-P 2.2.6 보안 위반 시 조치 관점에서, 컨테이너 보안 기준 위반(비-r...	Kubernetes PostgreSQL 컨테이너의 non-root 실행 강제 설정 부재	INFO
14	ISMS-P 2.5 인증 및 권한관리, 2.6 접근통제, 2.10 시스템 및 서비스 ...	Kubernetes 백엔드 컨테이너의 비루트 실행 보안 설정 누락	INFO
15	ISMS-P 관점에서 서비스 구성 요소의 접근통제 및 최소권한 원칙 미준수로 볼 수 있음...	Kubernetes 백엔드 컨테이너의 권한 상승 방지 설정 미흡	MEDIUM
16	ISMS-P 관점에서 안전한 시스템 운영을 위한 보호조치 미흡에 해당합니다. 특히 권...	Kubernetes 컨테이너가 non-root 실행 보안 설정 없이 정의됨	INFO
17	ISMS-P 관점에서는 운영환경의 안전한 설정 및 접근통제 미흡 가능성으로 볼 수 있음...	Kubernetes 컨테이너 보안 컨텍스트 미설정(검증 불가)	MEDIUM

18	ISMS-P 관점에서 안전한 기본 설정 미적용 및 최소권한 원칙 미준수에 해당할 수 ...	Kubernetes 컨테이너 비루트 실행 보안 설정 점검 미흡	INFO
19	ISMS-P 기술적 보호조치 측면에서 컨테이너 권한 상승 방지 통제가 미흡합니다. 주...	Kubernetes 컨테이너에 privilege escalation 차단 설정 누락	MEDIUM
20	ISMS-P 안내서 2.7 암호화 적용의 사례 4(비밀번호 찾기 등 개인정보 전송 구...	서버 비밀번호 재설정 로직 미확인으로 인한 취약점 판단 불가	INFO
21	ISMS-P의 접근통제 및 세션관리 관점에서, 클라이언트 저장소에 인증/식별 정보를 ...	추가 분석 불가: 로그인 프론트엔드 단독으로는 실제 비즈니스 로직 취약점 확정 어려움	INFO
22	ISMS-P의 비인가 접근 통제 및 비즈니스 로직 우회 방지 요구에 위배됩니다. 제로...	사용자 제공 API Key를 검증 없이 그대로 전달하는 외부 API 프록시로 인한 권한 통제 부재	HIGH
23	ISMS-P 인증기준의 접근통제 및 인증 관련 요구사항 위반 소지가 있으며, 계정 정...	비밀번호 변경 API에 사용자 식별자 (userId)를 클라이언트가 임의로 제출하여 IDOR/권한 우회 가능	HIGH
24	ISMS-P 웹 취약점 분석·평가 방법 상세가이드의 ‘권한 종류 및 프로세스 흐름을 ...	클라이언트 지정 userId로 작품 등록 권한을 우회할 수 있음	HIGH
25	ISMS-P 3.1.1 개인정보 수집·이용 취지 및 2.2.6 보안 위반 시 조치 관...	사용자 입력 OpenAI API Key를 서버로 전달하는 비즈니스 시크릿 오용	HIGH
26	ISMS-P 웹 취약점 분석·평가 항목의 세션/인증 관리 취지에 위배됩니다. 참조 1...	클라이언트 제어 userId를 이용한 작품 조회/수정/삭제로 인한 IDOR 가능성	HIGH
27	ISMS-P 관점에서 중앙 집중적 접근통제 및 최소권한 원칙(제로트러스트 원칙 나, ...	현재 코드만으로는 실제 접근통제 우회 취약점 확정 불가	INFO
28	ISMS-P 2.7 암호화 적용 및 계정정보 보호 관점에서, 비밀번호 변경과 같은 민...	비밀번호 변경 API에서 세션 사용자 검증이 없어 타 계정 비밀번호 변경 가능	HIGH
29	ISMS-P 2.7 암호화 적용에서 비밀번호는 안전한 방식으로 전달·처리되어야 하며,...	ID 찾기/비밀번호 찾기에서 계정 존재 여부가 노출되어 사용자 열거 가능	MEDIUM
30	U-13(안전한 비밀번호 암호화 알고리즘 사용), D-08(SHA-256 이상 해시 ...	현재 제공 코드만으로는 비밀번호 암호화 취약점 확정 불가	INFO

31	ISMS-P 2.7 암호화 적용 위반 소지. 참조 U-13(안전한 비밀번호 암호화 알...	로그인 인증 시 비밀번호를 평문 비 교하는 취약한 인증 로직	MEDIUM
32	ISMS-P 2.7 암호화 적용 및 접근 통제 관점 위반. 인증정보 변경 기능에서 명시...	비인증 사용자도 비밀번호 재설정이 가능한 권한 우회	HIGH
33	ISMS-P 관점에서 데이터 무결 성 보호 미흡. 동시성 제어 부재 로 동일 식별자 중복...	회원가입 시 이메일 중복 검사와 저 장 사이의 경쟁 상태 가능성	MEDIUM
34	ISMS-P 참조 U-13, D-08 및 2.7 암호화 적용 위반. 안전하지 않 은 비...	평문 비밀번호 저장 및 변경으로 인 한 계정정보 보호 실패	HIGH

2. Detailed Findings

2.1 코드 정적 분석 결과

1. [HIGH] Docker 컨테이너가 non-root 사용자 없이 실행되어 root 권한으로 동작할 수 있음

HIGH | TP | dockerfile.security.missing-user-entrypoint.missing-user-
entrypoint | mini_project5\Dockfile

Dockerfile의 최종 실행 단계에 USER 지시문이 없고, ENTRYPOINT만 정의되어 있어 컨테이너 내 Java 프로세스가 기본 root 권한으로 실행될 수 있습니다. 이 상태에서는 애플리케이션에 파일 경로 조작, 임의 파일 쓰기, 명령 실행 등의 취약점이 존재할 경우 컨테이너 장악 및 영향 범위 확대가 쉬워집니다. 제공된 스니펫상 비-root 전환이나 최소권한 보장이 없어 실제 취약점으로 판단합니다.

공격 시나리오: 공격자가 애플리케이션의 별도 취약점(예: 업로드 경로 조작, 명령 주입, 임의 파일 쓰기)을 이용해 서버 측 동작을 유도하면, 컨테이너 프로세스가 root로 실행 중이므로 /app 내부뿐 아니라 권한이 허용되는 범위의 시스템 파일/설정에 대한 추가 훼손이 가능해집니다. 결과적으로 컨테이너 내 민감정보 탈취, 악성 바이너리 실행, 서비스 변조로 이어질 수 있습니다.

취약 코드:

```
FROM amazoncorretto:17-alpine-jre
WORKDIR /app
COPY --from=builder /app/build/libs/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

수정 코드:

```
FROM amazoncorretto:17-alpine-jre
WORKDIR /app

RUN addgroup -S app && adduser -S -G app app
COPY --from=builder /app/build/libs/*.jar app.jar
RUN chown -R app:app /app

EXPOSE 8080
USER app
ENTRYPOINT ["java", "-jar", "app.jar"]
```

수정 가이드: 최종 실행 stage에 non-root 계정을 생성하고 USER app을 명시하여 애플리케이션이 루트 권한으로 실행되지 않도록 수정해야 합니다. 또한 /app 및 배포 산출물에 대한 소유권을 앱 사용자로 제한하여 최소권한 원칙을 적용하는 것이 바람직합니다.

ISMS-P: ISMS-P 관점에서 최소권한 원칙과 안전한 실행환경 관리에 미흡합니다. 컨테이너가 root로 실행되면 침해 시 영향 범위가 확대되며, OT/제로트러스트 관점의 '보호 대상을 작게 정의하고 권한을 최소화하는 점진적 적용' 원칙과도 상충합니다. 또한 보안 위반 시 조치(2.2.6) 관점에서 운영 표준에 비인가 root 실행을 금지하는 통제와 점검 절차를 두는 것이 필요합니다.

2. [HIGH] 하드코딩된 JWT 예시 토큰 문자열 사용

HIGH | FP | generic.secrets.security.detected-jwt-token.detected-jwt-token | mini_project5\book\frontend\src\app\userpage\view\page.js

소스코드에 JWT 형태의 문자열이 하드코딩되어 있어 비밀정보가 포함된 것처럼 보이지만, 제공된 문맥에서는 실제 인증에 사용되지 않습니다. 다만 운영 코드로 배포되거나 다른 위치에서 참조될 경우 세션/인증 토큰 노출 위험이 있으므로 제거하는 것이 바람직합니다.

오탐 사유: 탐지된 값은 FAKE_ACCESS_TOKEN이라는 이름의 상수에 하드코딩된 예시 JWT 문자열이며, 제공된 코드 구간에서는 이 값이 Authorization 헤더, 쿠키, 요청 본문 등 실제 인증 싱크로 전달되는 사용 경로가 확인되지 않습니다. 주석에도 '실제 accessToken으로 교체'라고 명시되어 있어 개발 중 임시 placeholder로 보입니다. 따라서 현재 스니펫만으로는 실제 노출/악용 가능한 취약점(TP)으로 단정할 수 없고, 세크릿 탐지 규칙에 의한 문자열 패턴 경보로 판단됩니다.

공격 시나리오: 현재 스니펫 기준으로는 공격 시나리오를 구성할 수 없습니다. 만약 이 상수가 다른 코드에서 Authorization 헤더로 사용된다면, 저장소 유출 또는 배포본 노출 시 예시 JWT가 외부에 노출될 수 있습니다.

수정 가이드: 하드코딩된 JWT 예시 문자열은 제거하고, 실제 인증 토큰은 로그인 후 서버 발급 값이나 안전한 세션 메커니즘을 통해 동적으로 주입해야 합니다. 클라이언트 소스에 비밀값을 남기지 말고, 토큰 만료·재발급·로그아웃 정책을 함께 적용해야 합니다.

ISMS-P: ISMS-P 웹 취약점 분석 항목 16(불충분한 세션 관리) 관점에서 JWT 등 인증 토큰은 안전한 서명 알고리즘, 비밀키 관리, 토큰 만료 정책이 요구됩니다. 다만 본 건은 실제 사용 중인 토큰 관리 취약점으로 확인되지 않았으므로 규정 위반으로 확정되지는 않습니다.

3. [HIGH] Dockerfile에서 실행 사용자(USER) 미지정으로 컨테이너가 root 권한으로 실행될 수 있음

HIGH | TP | dockerfile.security.missing-user-entrypoint.missing-user-entrypoint | mini_project5\book\Dockerfile

제공된 Dockerfile에는 USER 지시문이 없고, 마지막 실행 지점이 ENTRYPOINT ["java", "-jar", "app.jar"]로 되어 있어 컨테이너 기본 사용자(root)로 애플리케이션이 실행될 가능성이 높습니다. 이 경우 애플리케이션 취약점(RCE, 파일 쓰기,

경로 조작 등)이 발생하면 컨테이너 내부 권한이 과도하게 커져 영향 범위가 확대됩니다. 스니펫 상 root 실행을 배제하는 방어 코드가 확인되지 않아 실제 취약점으로 판단합니다.

공격 시나리오: 공격자가 애플리케이션의 별도 취약점(예: 파일 업로드 취약점, 명령 실행 취약점, 템플릿 인젝션 등)을 통해 컨테이너 내부에서 코드 실행에 성공하면, root 권한으로 실행 중인 프로세스를 통해 /app 이하 파일 수정, 민감 설정 파일 읽기, 로그/임시파일 변조, 추가 악성 바이너리 실행이 가능해집니다. 결과적으로 컨테이너 장악 및 서비스 무결성 훼손 위험이 커집니다.

취약 코드:

```
FROM public.ecr.aws/amazoncorretto/amazoncorretto:17-al2-jdk

WORKDIR /app
COPY build/libs/*.jar app.jar

EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

수정 코드:

```
FROM public.ecr.aws/amazoncorretto/amazoncorretto:17-al2-jdk

WORKDIR /app
COPY build/libs/*.jar app.jar

RUN useradd -m -u 10001 appuser \
    && chown -R appuser:appuser /app

USER 10001:10001
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

수정 가이드: 컨테이너 실행 사용자를 non-root로 명시하고, 애플리케이션 경로(/app) 및 산출물(app.jar)의 소유권을 해당 사용자로 변경해야 합니다. 베이스 이미지에 따라 useradd 사용이 어려우면 존재하는 non-root 계정(nobody 등)으로 전환하거나 멀티스테이지/권한 조정 방식으로 동일한 목표를 달성해야 합니다. 또한 런타임에 root 권한이 필요한 작업이 없도록 빌드 시점과 실행 시점을 분리하는 것이 좋습니다.

ISMS-P: ISMS-P 관점에서 최소권한 원칙 및 접근통제 강화 요구에 부합하지 않을 수 있습니다. 특히 컨테이너 실행 계정을 root로 두면 서비스 계정 권한을 최소화해야 한다는 운영 원칙에 어긋나며, 침해 시 영향 범위가 커집니다. 개인정보 처리 서비스라면 3.1.1 개인정보 수집·이용, 3.3.4 개인정보 국외이전 등과 연계된 데이터 보호 환경에서도 실행 권한 최소화가 필요한 기본 보안조치로 해석할 수 있습니다.

4. [MEDIUM] Kubernetes 컨테이너에 권한 상승 방지 설정 누락

~~MEDIUM | TP | yaml.kubernetes.security.allow-privilege-escalation=no-securitycontext.allow-privilege-escalation=no-securitycontext |~~
mini_project5\cluster-autoscaler-autodiscover.yaml

Deployment의 컨테이너에 securityContext.allowPrivilegeEscalation: false가 설정되어 있지 않아, 컨테이너 내부의 setuid/setgid 바이너리 또는 취약한 실행 흐름을 통해 권한 상승 가능성을 낮추는 방어가 누락되어 있습니다. Pod-level securityContext에 runAsNonRoot가 존재하더라도, 권한 상승 금지는 컨테이너 레벨에서 별도로 명시하는 것이 필요합니다.

공격 시나리오: 공격자가 컨테이너 내부에서 코드 실행에 성공하거나, 이미지 내 setuid/setgid 바이너리/프로세스 취약점을 악용하는 경우 allowPrivilegeEscalation 미설정 상태에서는 실행 프로세스의 권한 상승을 억제하지 못할 수 있습니다. 이후 컨테이너 권한을 이용해 민감 리소스 접근 범위를 넓히거나, 잘못 마운트된 볼륨/호스트 자원에 대한 추가 피해로 이어질 수 있습니다.

취약 코드:

```
spec:
  template:
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 65534
        fsGroup: 65534
      containers:
        - image: registry.k8s.io/autoscaling/cluster-autoscaler:v1.32.1
          name: cluster-autoscaler
```

수정 코드:

```
spec:
  template:
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 65534
        fsGroup: 65534
      containers:
        - name: cluster-autoscaler
          image: registry.k8s.io/autoscaling/cluster-autoscaler:v1.32.1
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities:
              drop:
... (1줄 생략)
```

수정 가이드: 컨테이너 레벨 securityContext를 추가하여

`allowPrivilegeEscalation: false`를 명시하고, 가능하면 `capabilities.drop: [ALL]`, `readOnlyRootFilesystem: true`도 함께 적용해 추가적인 권한 상승 경로를 차단하세요. Pod 레벨의 `runAsNonRoot`만으로는 권한 상승 방지가 완성되지 않으므로 컨테이너 단위 보안 설정을 보강해야 합니다.

ISMS-P: ISMS-P 관점에서 안전하지 않은 컨테이너 실행 정책은 접근통제 및 안전한 시스템 운영 원칙에 위배될 수 있으며, 침해 발생 시 개인정보의 유·노출 및 오·남용 위험을 키웁니다. 특히 권한 상승 방지 미흡은 보호조치 미흡으로 해석될 수 있어 내부 보안정책 준수 및 위반 시 조치(2.2.6) 체계와 연계한 점검·시정이 필요합니다.

5. [MEDIUM] 로그인 폼 조회용 `RequestMapping`의 HTTP Method 미지정

MEDIUM | FP | `java.spring.security.unrestricted-request-mapping.unrestricted-request-mapping | mini_project5\book\src\main\java\com\aiivle0102\book\controller\UserController`

`UserController`의 `/login/form` 엔드포인트는 로그인 화면 렌더링용 조회 요청으로 보이며, 세션을 읽기만 하고 변경하지 않습니다. HTTP method 미지정 자체는 권장되지 않지만, 본 코드만으로는 CSRF로 인한 실제 피해가 발생하는 상태 변경 동작이 확인되지 않아 실취약점으로 보기 어렵습니다.

오탐 사유: 해당 핸들러는 `@RequestMapping("/login/form")`로 HTTP method가 명시되지 않았지만, 본문은 세션 속성 조회 후 로그인 화면을 반환하거나 이미 로그인된 경우 리다이렉트하는 조회성 동작만 수행합니다. `setAttribute`, `invalidate`, DB 변경, 계정/권한 변경 등 상태 변경이 없으므로 CSRF의 전제가 되는 '상태 변경 요청'이 성립하지 않습니다. 따라서 이 탐지는 현재 코드 문맥상 오탐으로 판단됩니다.

공격 시나리오: 공격자가 `/login/form`에 대해 GET/POST 등 임의 메서드로 요청을 보내도 서버 측 상태가 변경되지 않습니다. 따라서 CSRF 관점의 실제 영향은 없고, 단지 로그인 화면이 반환되거나 세션 존재 시 `/main`으로 리다이렉트될 뿐입니다.

수정 가이드: 로그인 화면 조회용 엔드포인트임을 명확히 하기 위해 `@GetMapping`으로 제한하는 것이 좋습니다. 기능상 상태 변경이 없다면 CSRF 취약점은 아니지만, HTTP method를 명시하여 오동작 가능성과 보안 점검 시 혼선을 줄일 수 있습니다.

ISMS-P: ISMS-P 관점에서 세션 관리 정책(C-14, 불충분한 세션 관리/세션 타임아웃)이나 세션 보호 원칙을 점검하는 것이 중요하지만, 본 코드에서는 세션 만료 미설정, 세션 고정, 예측 가능한 세션 ID 생성 등과 같은 실제 위반 정황이 확인되지 않았습니다.제로트러스트 원칙상 '명시적 신뢰 후 접근' 및 '세션 단위 접근 허용'을 고려하더라도, 여기서는 세션 읽기만 수행하므로 위반으로 단정할 수 없습니다.

6. [MEDIUM] Kubernetes 컨테이너에 `privilege escalation` 차단 설정 누락

MEDIUM | TP | `yaml.kubernetes.security.allow-privilege-escalation-no-`

```
securitycontext.allow-privilege-escalation=no-securitycontext |  
mini_project5\k8s-deployment.yaml
```

backend 컨테이너에 securityContext가 정의되어 있지 않아 allowPrivilegeEscalation 이 기본 허용 상태로 동작할 수 있습니다. 컨테이너 이미지 내부에 setuid/setgid 바이너리 또는 권한 상승에 유리한 구성이 존재할 경우, 공격자가 컨테이너 내부 실행 권한을 확보했을 때 추가 권한 상승을 시도할 수 있습니다. 이는 컨테이너 격리와 최소권한 원칙을 약화시키므로 명시적으로 privilege escalation을 차단해야 합니다.

공격 시나리오: 공격자가 애플리케이션 취약점(RCE, 파일 업로드 악용 등)으로 backend 컨테이너 내부에서 셸 실행 권한을 얻은 뒤, 이미지 내 setuid 바이너리나 권한 상승 가능한 명령을 이용해 상위 권한으로 상승을 시도합니다. allowPrivilegeEscalation이 차단되지 않으면 이러한 공격 성공 가능성이 높아집니다.

취약 코드:

```
160:         containers:  
161:             - name: backend  
162:               image: 416225317325.dkr.ecr.us-east-1.amazonaws.com/ai0  
163:               ports:  
164:                 - containerPort: 8080  
165:               resources:  
166:                 requests:  
167:                   cpu: "100m"  
168:                   memory: "256Mi"  
169:                 limits:  
170:                   cpu: "500m"  
171:                   memory: "512Mi"  
172:               env:
```

수정 코드:

```
containers:  
  - name: backend  
    image: 416225317325.dkr.ecr.us-east-1.amazonaws.com/ai0102-b  
    securityContext:  
      allowPrivilegeEscalation: false  
      runAsNonRoot: true  
      readOnlyRootFilesystem: true  
      capabilities:  
        drop: ["ALL"]  
    ports:  
      - containerPort: 8080  
    resources:  
      requests:  
        cpu: "100m"  
        memory: "256Mi"  
... (3줄 생략)
```

수정 가이드: backend 컨테이너에 securityContext를 추가하여 privilege escalation을

명시적으로 차단하고, 가능하면 바루트 실행, 읽기 전용 루트 파일시스템, 모든 Linux capability 제거를 함께 적용하십시오. initContainer 및 다른 워크로드에도 동일한 정책을 일관되게 적용하면 최소권한 원칙과 제로트러스트(신뢰하지 말고 항상 검증) 원칙에 부합합니다.

ISMS-P: 최소권한 원칙 및 안전한 실행환경 통제 미흡으로, ISMS-P 관점에서 개인정보 처리 시스템의 안전성 확보조치가 부족합니다. 특히 컨테이너 내부 권한 상승이 발생할 경우 개인정보 저장소/DB 접속정보 노출로 이어질 수 있어, 개인정보 보호를 위한 접근통제 및 실행환경 보호 요구사항에 위배될 소지가 있습니다.

7. [MEDIUM] Kubernetes 컨테이너에 권한 상승 방지 설정이 누락됨

MEDIUM | TP | `yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext | mini_project5\k8s-deployment.yaml`

StatefulSet의 postgres 컨테이너 정의에 securityContext가 없고 allowPrivilegeEscalation: false도 설정되어 있지 않습니다. Kubernetes에서 해당 값이 명시되지 않으면 컨테이너 내부 setuid/setgid 바이너리나 취약한 런타임 조건을 통해 권한 상승 가능성이 커집니다. 본 구성은 최소권한 원칙에 맞지 않으며, 컨테이너 침해 시 호스트/볼륨/서비스 계정 자원으로의 추가 확장 위험이 있습니다.

공격 시나리오: 공격자가 postgres 컨테이너 내부에서 명령 실행 권한을 획득한 뒤, allowPrivilegeEscalation 제한이 없어 setuid/setgid 바이너리 또는 권한상승 가능한 경로를 이용해 root 권한으로 상승합니다. 이후 DB 데이터 볼륨, 쿠버네티스 서비스 계정 토큰, 인접 네트워크 자원에 접근하여 민감정보 탈취 또는 추가 침투를 수행할 수 있습니다.

취약 코드:

```
82         spec:
83             containers:
84                 - name: postgres
85                   image: postgres:15-alpine
86                   ports:
87                     - containerPort: 5432
88                   env:
```

수정 코드:

```
spec:
  containers:
    - name: postgres
      image: postgres:15-alpine
      securityContext:
        allowPrivilegeEscalation: false
        runAsNonRoot: true
        capabilities:
          drop: ["ALL"]
```

```

    ports:
      - containerPort: 5432
    env:
      - name: POSTGRES_DB
        value: bookdb
      - name: POSTGRES_USER
... (9줄 생략)

```

수정 가이드: 해당 컨테이너에 securityContext를 추가하고 allowPrivilegeEscalation을 false로 고정해야 합니다. 가능하다면 runAsNonRoot와 capability drop도 함께 적용하여 최소권한 원칙을 강화하십시오. postgres 이미지 특성상 추가 권한이 필요한지 배포 환경에서 검증한 뒤, 필요한 경우 예외를 최소화한 채로 조정해야 합니다.

ISMS-P: ISMS-P의 최소권한 및 접근통제 취지에 부합하지 않으며, 컨테이너 보안 설정 미흡으로 인해 침해 시 권한상승 가능성을 키웁니다. 주요정보통신기반시설 가이드의 보안 설정 항목 및 OT Zero Trust 원칙(기본 불신, 최소권한 적용)과도 상충합니다.

8. [MEDIUM] Kubernetes 컨테이너 권한 상승 방지 설정 누락

MEDIUM | TP | `yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext | mini_project5\k8s-deployment.yaml`

Deployment의 nginx 컨테이너에 securityContext가 정의되어 있지 않아 allowPrivilegeEscalation=false가 적용되지 않았습니다. 이 설정 누락은 컨테이너 내부에서 setuid/setgid 바이너리 또는 취약한 실행 경로가 존재할 경우 권한 상승 가능성을 확대합니다. Kubernetes 보안 기준상 컨테이너 권한 상승을 원천 차단하는 방어 구성이 미흡한 상태입니다.

공격 시나리오: 공격자가 nginx 컨테이너 내에서 다른 취약점(RCE, 파일 업로드 악용, 취약한 모듈 등)을 통해 셸을 획득한 뒤, 컨테이너에 setuid/setgid 바이너리나 권한 상승 가능한 동작이 존재하면 루트 권한으로 상승을 시도할 수 있습니다. 이 경우 동일 파드 내 민감 파일 접근, 토큰 탈취, 클러스터 측 이동(lateral movement) 위험이 커집니다.

취약 코드:

```

containers:
  - name: nginx
    image: nginx:alpine
    ports:
      - containerPort: 80
    readinessProbe:
      tcpSocket:
        port: 80

```

수정 코드:

```

containers:

```

```

- name: nginx
  image: nginx:alpine
  securityContext:
    allowPrivilegeEscalation: false
    runAsNonRoot: true
    readOnlyRootFilesystem: true
    capabilities:
      drop: ["ALL"]
  ports:
    - containerPort: 80
  readinessProbe:
    tcpSocket:
      port: 80

```

수정 가이드: nginx 컨테이너에 securityContext를 추가하여 권한 상승을 금지하고, 가능한 경우 비루트 실행과 불필요 capability 제거를 함께 적용해야 합니다.

initContainer에도 동일한 하드닝을 적용하면 더 안전합니다. 이는 ISMS-P 2.8.1(보안 요구사항 정의) 및 2.10.2(클라우드 보안) 관점에서 배포 시 필수 보안 요구사항을 설계 단계부터 반영하는 조치입니다.

ISMS-P: ISMS-P 2.8.1 보안 요구사항 정의 미흡에 해당합니다. 컨테이너 배포 시 최신 보안 취약점 및 안전한 구현 요구사항으로 allowPrivilegeEscalation=false 같은 하드닝을 명시해야 하나, 해당 방어 설정이 누락되었습니다. 클라우드 기반 서비스의 안전한 운영 관점에서는 2.10.2 클라우드 보안 취지에도 부합하지 않습니다.

9. [MEDIUM] Frontend 컨테이너의 권한 상승 방지 설정 누락

MEDIUM | TP | `yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext | mini_project5\k8s-deployment.yaml`

Kubernetes Deployment의 frontend 컨테이너에 securityContext가 정의되어 있지 않아 allowPrivilegeEscalation=false가 보장되지 않습니다. 이로 인해 이미지 내부의 setuid/setgid 바이너리 또는 런타임 환경을 악용한 권한 상승 가능성이 커질 수 있습니다. 제공된 스니펫 상 별도의 방어 로직이나 상위 제약이 확인되지 않아 취약한 보안 설정으로 판단됩니다.

공격 시나리오: 공격자가 애플리케이션 취약점 또는 탈취한 일반 계정으로 frontend 컨테이너 내부에 진입한 뒤, setuid/setgid 바이너리나 허술한 런타임 설정을 악용해 권한을 상승시킬 수 있습니다. 이후 동일 Pod 내 민감 자원 접근, 호스트 마운트 경로 탐색, 자격 증명 탈취 등의 후속 공격으로 이어질 수 있습니다.

취약 코드:

```

containers:
- name: frontend
  image: 416225317325.dkr.ecr.us-east-1.amazonaws.com/ai0102-frontend:latest
  ports:

```

```

    - containerPort: 3000
  resources:
    requests:
      cpu: "100m"
      memory: "256Mi"
    limits:
      cpu: "500m"
      memory: "512Mi"

```

수정 코드:

```

containers:
  - name: frontend
    image: 416225317325.dkr.ecr.us-east-1.amazonaws.com/ai0102-front:latest
    ports:
      - containerPort: 3000
    securityContext:
      allowPrivilegeEscalation: false
      runAsNonRoot: true
      readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]
    resources:
      requests:
        cpu: "100m"
        memory: "256Mi"
... (3줄 생략)

```

수정 가이드: frontend 컨테이너에 securityContext를 명시하고 allowPrivilegeEscalation을 false로 설정하여 프로세스 권한 상승을 차단해야 합니다. 가능하다면 runAsNonRoot, readOnlyRootFilesystem, capabilities.drop: ['ALL']도 함께 적용해 컨테이너 탈취 시 영향 범위를 최소화하세요.

ISMS-P: ISMS-P 기술적 보호조치 측면에서 컨테이너 권한 상승 방지 설정이 미흡하여 최소권한 원칙과 보안 구성 관리 요구사항에 부합하지 않습니다. 특히 주요정보통신기반시설 기술적 취약점 분석·평가 가이드의 권한 상승 방지 관점과, 보안 위반 시 내부 통제 및 조치 체계(2.2.6) 관점에서 취약한 구성으로 평가될 수 있습니다.

10. [MEDIUM] Kubernetes 컨테이너 권한상승 방지 설정 미확인

MEDIUM | TP | N/A | mini_project5\k8s-minikube.yaml

Semgrep 규칙은 컨테이너에 securityContext.allowPrivilegeEscalation=false 설정이 없거나 확인되지 않을 때 탐지합니다. 현재는 파일 원문을 확인하지 못해 postgres 컨테이너에 해당 설정이 실제로 존재하는지 검증할 수 없습니다.

11. [MEDIUM] Kubernetes 백엔드 컨테이너의 권한 상승 방지 설정 미흡

```
MEDIUM | TP | yam1.kubernetes.security.allow-privilege-escalation=no-  
securitycontext.allow-privilege-escalation-no-securitycontext |  
mini_project5\k8s-minikube.yaml
```

k8s-minikube.yaml의 backend Deployment 컨테이너에 securityContext가 없고 allowPrivilegeEscalation=false가 명시되지 않아, 컨테이너 내부 setuid/setgid 바이너리 또는 취약한 실행 경로를 통해 권한 상승이 가능해질 수 있습니다. 이 문제는 입력 값 기반 취약점이 아니라 컨테이너 실행 정책 누락에 해당하므로 Semgrep 탐지는 타당합니다.

공격 시나리오: 공격자가 backend 애플리케이션 RCE 또는 컨테이너 내부 셸을 획득한 경우, allowPrivilegeEscalation이 허용된 상태에서는 setuid/setgid 프로그램을 이용한 권한 상승 시도가 가능해집니다. 이후 컨테이너 내 민감 데이터(예: DB 연결 정보, 마운트된 볼륨) 접근 범위가 확대될 수 있습니다.

취약 코드:

```
containers:  
  - name: backend  
    image: backend:latest  
    imagePullPolicy: Never  
    ports:  
      - containerPort: 8080  
    resources:  
      requests:  
        cpu: "100m"  
        memory: "256Mi"  
      limits:  
        cpu: "500m"  
        memory: "512Mi"
```

수정 코드:

```
containers:  
  - name: backend  
    image: backend:latest  
    imagePullPolicy: Never  
    securityContext:  
      allowPrivilegeEscalation: false  
      runAsNonRoot: true  
      readOnlyRootFilesystem: true  
      capabilities:  
        drop: ["ALL"]  
    ports:  
      - containerPort: 8080  
    resources:  
      requests:  
        cpu: "100m"  
... (4줄 생략)
```

수정 가이드: backend 컨테이너에 securityContext를 추가하여 권한 상승을 원천 차단해야 합니다. 최소한 allowPrivilegeEscalation:false를 적용하고, 가능한 경우 runAsNonRoot, readOnlyRootFilesystem, capabilities drop ALL까지 함께 적용해 컨테이너 침해 시 피해 범위를 줄이세요. 적용 후 애플리케이션 동작에 영향이 있는지 검증해야 합니다.

ISMS-P: ISMS-P 관점에서 서비스 구성요소의 접근통제 및 최소권한 원칙 미준수로 볼 수 있습니다. 특히 컨테이너 권한상승 방지 설정 부재는 내부자/침해자 권한 확대 위험을 높여 개인정보 보호를 위한 기술적·관리적 보호조치 취약점에 해당합니다. OT/제로트러스트 관점에서도 보호 대상을 작은 단위로 분리하고(참조 3), 각 실행 단위의 최소 권한을 강제해야 하는 원칙에 어긋납니다.

12. [MEDIUM] Kubernetes 컨테이너 보안 컨텍스트 미설정(검증 불가)

```
MEDIUM | FP | yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext | mini_project5\k8s-minikube.yaml
```

Semgrep는 frontend Deployment의 컨테이너 정의에서

securityContext.allowPrivilegeEscalation: false 부재 가능성을 경고했지만, 현재 제공된 코드 조각만으로는 해당 설정이 실제로 누락되었는지 단정할 수 없습니다. 따라서 현 시점에서는 오탐 가능성이 더 높으며, 전체 YAML 문맥 확인이 선행되어야 합니다.

오탐 사유: 제공된 스니펫만으로는 frontend 컨테이너에 securityContext가 실제로 누락되었는지 확인할 수 없습니다. 현재 보이는 범위는 컨테이너 선언과 자원 제한(resource requests/limits)까지만이며, allowPrivilegeEscalation 값이 존재하는지 여부는 상위/하위 문맥이 더 필요합니다. 또한 이 경고는 Kubernetes 배포 설정의 하드닝 권고에 가깝고, 제공된 맥락상 공격자 입력이 직접 실행 경로로 이어지는 취약점 흐름(명시적 사용자 제어 입력 → 실행부)이 증명되지 않았습니다.

공격 시나리오: 만약 해당 컨테이너가 실제로 securityContext 없이 실행되고, 이미지 내부에 setuid/setgid 바이너리 또는 권한 상승 가능한 실행 파일이 존재한다면, 컨테이너 내 침해 후 로컬 권한 상승이 가능할 수 있습니다. 그러나 현재 문맥에서는 그 조건이 확인되지 않아 실질 공격 시나리오로 확정할 수 없습니다.

수정 가이드: 컨테이너 단위 securityContext를 명시하여 권한 상승을 차단하고, 비루트 실행 및 기본 seccomp 프로파일을 함께 적용하십시오. 이는 Kubernetes 컨테이너 하드닝의 기본 조치로, 런타임 침해 시 로컬 권한 상승과 시스템 자원 접근 범위를 줄이는 데 도움이 됩니다.

ISMS-P: ISMS-P 관점에서는 운영환경의 안전한 설정 및 접근통제 미흡 가능성으로 볼 수 있으나, 현재 문맥상 개인정보 수집·이용(3.1.1)이나 국외이전(3.3.4)과의 직접 위반 연결은 확인되지 않습니다. 다만 안전한 구성·권한 최소화 원칙 위반 소지가 있어 운영 보안 통제 측면에서 보완이 필요합니다.

13. [MEDIUM] Kubernetes 컨테이너에 **privilege escalation** 차단 설정 누락

MEDIUM | TP | `yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext` |
`mini_project5\k8s-minikube.yaml`

k8s-minikube.yaml의 nginx 컨테이너 정의에 securityContext가 없어서 allowPrivilegeEscalation=false가 명시되지 않았습니다. Kubernetes에서 기본적으로 컨테이너 내부의 setuid/setgid 바이너리나 권한 상승 경로가 남아 있으면, 공격자가 컨테이너 권한을 상향시켜 마운트된 자원, 서비스 토큰, 민감 파일에 접근할 수 있습니다. 제공된 문맥상 상위 Pod 레벨 securityContext 상속이나 별도 차단 로직이 확인되지 않아, Semgrep 경고는 실제 하드닝 미흡으로 판단됩니다. ISMS-P 관점에서도 사용자 입력/공격 실행 이후 권한 상승 가능성을 차단하는 기술적 통제가 부족한 상태로 볼 수 있습니다.

공격 시나리오: 공격자가 애플리케이션 취약점(예: 업로드 취약점, RCE, SSRF 등)으로 nginx 컨테이너 내부 셸 또는 코드 실행 권한을 얻은 뒤, allowPrivilegeEscalation이 허용된 상태를 악용해 setuid/setgid 바이너리나 권한 상승 가능한 로직을 실행하여 더 높은 권한의 프로세스로 전환합니다. 이후 컨테이너 내 민감 설정, 마운트 볼륨, 토큰, 네트워크 접근 범위를 확대해 추가 침투나 정보 탈취를 수행할 수 있습니다.

취약 코드:

```
272         spec:
273         initContainers:
274             - name: wait-for-frontend
275               image: busybox:1.28
276               command: ['sh', '-c', "until nslookup frontend; do echo
277         containers:
278             - name: nginx
279               image: nginx:alpine
280               ports:
281                 - containerPort: 80
282               readinessProbe:
283                 tcpSocket:
284                   port: 80
285                 initialDelaySeconds: 10
286                 periodSeconds: 5
287         ... (7줄 생략)
```

수정 코드:

```
spec:
  template:
    spec:
      containers:
        - name: nginx
          image: nginx:alpine
```

```
securityContext:
  allowPrivilegeEscalation: false
  runAsNonRoot: true
  readOnlyRootFilesystem: true
  capabilities:
    drop: ["ALL"]
  ports:
    - containerPort: 80
  readinessProbe:
... (11줄 생략)
```

수정 가이드: nginx 컨테이너에 securityContext를 추가하여 privilege escalation을 명시적으로 차단해야 합니다. allowPrivilegeEscalation=false를 설정하고, 가능하면 runAsNonRoot, readOnlyRootFilesystem, capabilities.drop:[ALL]까지 함께 적용해 컨테이너 탈취 이후의 권한 상승 및 공격면을 줄이십시오. 필요 시 initContainer에도 동일한 정책을 검토하고, 변경 후 Semgrep/Kube Bench 등으로 재점검하는 것이 좋습니다.

ISMS-P: ISMS-P 기술적 보호조치 측면에서 컨테이너 권한 상승 방지 통제가 미흡합니다. 주요정보통신기반시설 기술적 취약점 분석·평가 방법 상세가이드의 Web Application 보안/코드 인젝션 항목에서 강조하는 바와 같이, 비인가 코드 실행 및 권한 상승 가능성을 차단해야 하며, Kubernetes에서는 allowPrivilegeEscalation=false와 최소권한 원칙(제로트러스트/Least Privilege) 적용이 요구됩니다.

14. [INFO] Kubernetes Pod 컨테이너의 Non-Root 실행 설정 누락

INFO | TP | `yaml.kubernetes.security.run-as-non-root.run-as-non-root` | `mini_project5\k8s-deployment.yaml`

StatefulSet의 Pod 템플릿(spec.template.spec) 하위 컨테이너(postgres)에 runAsNonRoot 또는 대체 non-root 보안 설정이 명시되지 않아, 컨테이너가 root 권한으로 실행될 가능성이 있습니다. 이는 Kubernetes 배포 보안 기준상 권한 상승 및 컨테이너 탈출/피해 확대 위험을 높이는 구성 취약점입니다. 제공된 스니펫에서는 상위/하위 securityContext로 non-root 실행을 강제하는 방어 로직이 확인되지 않았습니다.

15. [INFO] Kubernetes 컨테이너의 비루트 실행 설정 누락 여부 확인 불가

INFO | FP | `yaml.kubernetes.security.run-as-non-root.run-as-non-root` | `mini_project5\k8s-deployment.yaml`

현재 제공된 코드 조각만으로는 해당 Deployment가 실제로 root 권한으로 실행되는지, 또는 상위 수준(Pod/Container SecurityContext)에서 이미 runAsNonRoot: true가 적용되어 있는지 검증할 수 없습니다. 따라서 이 탐지 결과는 하드닝 권고 수준의 정보성 경고이며, 실제 취약점(TP)으로 확정할 근거가 부족합니다.

16. [INFO] Kubernetes 컨테이너 비-root 실행 보안 설정 누락

INFO | TP | `yaml.kubernetes.security.run-as-non-root.run-as-non-root` |
`mini_project5\k8s-deployment.yaml`

Deployment의 backend Pod 스펙(라인 148 이후)에서 컨테이너 실행 사용자에 대한 보안 컨텍스트가 명시되지 않았습니다. 특히 `runAsNonRoot: true`, `runAsUser`, `allowPrivilegeEscalation` 차단, `capabilities drop` 등의 제한이 없어 컨테이너가 root로 실행될 가능성을 배제할 수 없습니다. 이는 컨테이너 내부 취약점이 발생했을 때 권한 상승 및 마운트된 볼륨/민감 자원 접근 범위를 확대할 수 있어 보안상 미흡합니다. ISMS-P 관점에서는 비인가 권한 사용을 제한하는 보안 통제 미흡으로 볼 수 있으며, UNIX 계정/권한 관리 원칙(U-63 취지)과도 부합하는 하드닝 조치가 필요합니다.

17. [INFO] Kubernetes PostgreSQL 컨테이너의 non-root 실행 강제 설정 부재

INFO | TP | `yaml.kubernetes.security.run-as-non-root.run-as-non-root` |
`mini_project5\k8s-minikube.yaml`

StatefulSet의 postgres 컨테이너에 `securityContext.runAsNonRoot: true`가 설정되어 있지 않아, 컨테이너가 root 권한으로 실행될 가능성을 구성 차원에서 배제할 수 없습니다. 이는 컨테이너 내 취약점 악용 시 권한 상승 및 호스트/볼륨 접근 영향 범위를 확대할 수 있으므로, Kubernetes 보안 기준상 non-root 실행을 명시적으로 강제해야 합니다.

18. [INFO] Kubernetes 컨테이너가 root 권한으로 실행될 수 있음

INFO | TP | N/A | `mini_project5\k8s-deployment.yaml`

19. [INFO] Kubernetes 백엔드 컨테이너의 비루트 실행 보안 설정 누락

INFO | TP | `yaml.kubernetes.security.run-as-non-root.run-as-non-root` |
`mini_project5\k8s-minikube.yaml`

해당 Deployment의 backend 컨테이너에 `securityContext.runAsNonRoot: true`가 명시되어 있지 않아, 컨테이너가 root 권한으로 실행될 가능성이 있습니다. 제공된 스니펫 기준으로는 Pod/Container 레벨의 대체 보안 설정도 확인되지 않으므로, Semgrep 경고는 실제 보안 개선 필요 사항으로 판단됩니다. 이 설정 누락은 컨테이너 탈취 시 권한 상승 및 호스트/클러스터 자원 접근 위험을 확대할 수 있습니다.

20. [INFO] Kubernetes 컨테이너가 non-root 실행 보안 설정 없이 정의됨

INFO | TP | `yaml.kubernetes.security.run-as-non-root.run-as-non-root` |
`mini_project5\k8s-minikube.yaml`

Frontend Deployment의 Pod template 내 컨테이너 정의에

`securityContext.runAsNonRoot: true`가 명시되어 있지 않습니다. 이 경우 이미지의 기본 사용자 설정에 따라 컨테이너가 root 권한으로 실행될 수 있으며, 침해 시 호스트/볼륨/클러스터 리소스에 대한 영향 범위가 커질 수 있습니다. Semgrep 규칙은 이러한 기본 하드닝 누락을 탐지한 것으로 판단되며, 제공된 스니펫 기준으로 오탐으로 불균거는 확인되지 않았습니다.

21. [INFO] Kubernetes 컨테이너 비루트 실행 보안 설정 점검 미흡

INFO | FP | `yaml.kubernetes.security.run-as-non-root.run-as-non-root | mini_project5\k8s-minikube.yaml`

Kubernetes Pod/Container에 `runAsNonRoot: true`가 명시되지 않으면 컨테이너가 root로 실행될 가능성이 있어 권한 상승 시 피해가 확대될 수 있습니다. 다만 본 케이스는 현재 스니펫만으로는 실제 배포 설정 전체를 검증할 수 없어 취약점 확정이 불가능합니다.

2.2 비즈니스 로직 심층 분석 결과

1. [HIGH] 사용자 제공 API Key를 검증 없이 그대로 전달하는 외부 API 프록시로 인한 권한 통제 부재

HIGH | Broken Access Control | `mini_project5\book\frontend\src\app\api\generateCover\route.js`

이 라우트는 요청 본문에서 받은 `apiKey`를 그대로 OpenAI 호출의 Bearer 토큰으로 사용합니다. 서버 측에서 호출 주체의 인증/인가를 확인하지 않고, 어떤 키를 사용할지에 대한 정책도 없습니다. 그 결과 비인가 사용자가 이 엔드포인트를 통해 임의의 API Key로 이미지 생성 요청을 중계할 수 있어, 서비스의 비용 오남용 및 기능 악용이 가능합니다. 이는 제로트러스트 관점에서 '모든 접근은 기본 거부 후 명시적 검증' 원칙과, ISMS-P의 비인가 접근 통제 및 비즈니스 로직 우회 방지 요구에 위배됩니다.

공격 시나리오: 공격자가 이 API에 직접 요청을 보내 `apiKey`와 `prompt`를 임의로 조작하면, 인증되지 않은 상태에서도 서버를 OpenAI 프록시처럼 사용하여 이미지 생성 요청을 수행할 수 있습니다. 이로 인해 조직이 의도하지 않은 비용 발생, 기능 남용, 사용자별 사용 제한 우회가 가능합니다.

취약 코드:

```
const { apiKey, model, prompt } = await req.json();
...
headers: {
  "Authorization": `Bearer ${apiKey}`,
  "Content-Type": "application/json"
}
```

수정 코드:

```
import { NextResponse } from "next/server";

export async function POST(req) {
  try {
    // 1) 사용자 인증/인가 확인
    // 2) 서버 환경변수에 저장된 OpenAI 키만 사용
    const { model, prompt } = await req.json();
    const openaiApiKey = process.env.OPENAI_API_KEY;

    if (!openaiApiKey) {
      return NextResponse.json({ error: "Server API key not configured" },
    }

    const openaiRes = await fetch("https://api.openai.com/v1/images/genera
      method: "POST",
    ... (22줄 생략)
```

수정 가이드: 클라이언트가 전달한 API Key를 신뢰하지 말고, 서버가 보관한 비밀값만 사용하도록 변경해야 합니다. 또한 이 기능은 로그인 사용자에게만 허용하고, 사용자 별 쿼터·rate limit·감사 로그를 적용하여 비인가 호출과 비용 남용을 차단해야 합니다.

ISMS-P: ISMS-P의 비인가 접근 통제 및 비즈니스 로직 우회 방지 요구에 위배됩니다. 제로트러스트 원칙(모든 접근에 대해 기본 거부, 중앙 집중적 정책 관리, 최소 권한)에 반하며, Web 취약점 점검 기준의 "권한 검증 로직 미구현/중간 단계 우회" 사례(U-권한 통제 항목에 해당하는 유형)와 유사합니다.

2. [HIGH] 비밀번호 변경 API에 사용자 식별자(userId)를 클라이언트가 임의로 제출하여 IDOR/권한 우회 가능

HIGH | IDOR | mini_project5/book/frontend/src/app/password/change/page.js

프론트엔드가 localStorage의 user 객체에서 userId를 읽어 그대로 /api/user/change-pw 요청 본문에 포함합니다. 이 구조에서는 공격자가 브라우저에서 localStorage.user.userId 값을 임의의 계정으로 바꾼 뒤 비밀번호 변경을 시도할 수 있습니다. 서버가 요청자의 세션/토큰 주체와 body의 userId를 강제 매칭하지 않으면 타 계정 비밀번호 변경이 가능한 Broken Access Control(IDOR)로 이어집니다. ISMS-P 관점에서는 인증/권한 검증의 적정성 미흡(권한 없는 변경 가능)으로, 개인정보 및 계정정보 변경 시 서버측 주체 검증이 필수입니다.

공격 시나리오: 공격자는 로그인 후 개발자도구에서 localStorage의 user.userId를 피해자 계정 ID로 변경한다. 이후 현재 비밀번호 검증 로직이 서버에서 대상 계정 기준으로 느슨하게 처리되거나, userId만 신뢰하는 경우 피해자 계정의 비밀번호를 공격자가 아는 currentPassword(또는 검증 우회 값)로 변경할 수 있다. 최소한 계정 존재 여부/오류 차이를 통해 계정 열거도 가능하다.

취약 코드:

```
setUserId(user.userId); // 로그인 때 저장해 둔 userId 사용
...
body: JSON.stringify({
  userId,
  currentPassword,
  newPassword,
}),
```

수정 코드:

```
const res = await fetch('/api/user/change-pw', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    currentPassword,
    newPassword,
  }),
});

// 서버에서는 세션/토큰의 subject로 대상 계정을 결정하고,
// body로 전달된 userId는 무시하거나 세션 값과 일치하는지 검증
```

수정 가이드: 클라이언트가 `userId`를 전달하지 않도록 변경하고, 서버에서 인증된 세션/토큰의 주체를 기준으로 대상 계정을 식별해야 합니다. 또한 현재 비밀번호 검증과 비밀번호 변경은 반드시 서버에서 원자적으로 처리하고, 요청 본문 값만으로 권한을 결정하지 않도록 수정해야 합니다.

ISMS-P: ISMS-P 인증기준의 접근통제 및 인증 관련 요구사항 위반 소지가 있으며, 계정 정보 변경 시 요청자와 대상자 식별을 서버에서 검증하지 않으면 권한 없는 변경을 허용할 수 있습니다. 이는 인증/권한 부여 절차의 무결성 미흡에 해당합니다.

3. [HIGH] 클라이언트 지정 `userId`로 작품 등록 권한을 우회할 수 있음

HIGH | IDOR |

mini_project5\book\frontend\src\app\userpage\create\page.js

작품 등록 요청 시 `userId`를 URL 쿼리스트링으로 클라이언트가 직접 지정합니다. 프론트는 `localStorage`의 사용자 정보를 신뢰하고 있으며, 서버가 이 값을 인증 주체와 재검증하지 않으면 공격자가 개발자도구/프록시로 `userId`를 임의 변경해 타 사용자 계정 명의로 작품을 등록하는 IDOR/권한 우회가 가능합니다. ISMS-P 웹 취약점 분석 가이드의 '권한 종류 및 프로세스 흐름을 파악하고 통제를 우회 및 악용 가능성 점검' 항목에 해당하며, 서버 사이드에서 세션/토큰 기반 접근 통제를 강제해야 합니다.

공격 시나리오: 공격자가 브라우저에서 `localStorage.user.userId`를 다른 사용자 ID로 바꾸거나, 요청 직전에 프록시로 `userId` 값을 수정한 뒤 작품 등록을 수행합니다. 백엔드가 세션 기반 검증 없이 해당 `userId`를 소유자/작성자로 사용하면 공격자는 타 사용자

계정에 콘텐츠를 생성하거나, 권한이 필요한 등록 행위를 우회할 수 있습니다.

취약 코드:

```
const userData = JSON.parse(localStorage.getItem("user"));
...
const response = await fetch(`/api/book/insertByUrl?userId=${userData.user
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(bookData),
});
```

수정 코드:

```
const response = await fetch(`/api/book/insertByUrl`, {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization": `Bearer ${accessToken}`
  },
  body: JSON.stringify(bookData),
});

// 서버에서는 인증 토큰/세션의 사용자만 사용
const authenticatedUserId = req.user.id;
bookService.createBook(authenticatedUserId, bookData);
```

수정 가이드: 소유자 식별자(userId)를 클라이언트 입력값으로 받지 말고, 서버가 인증된 세션/토큰의 주체를 기준으로 강제 지정해야 합니다. 또한 요청 객체의 userId가 존재하더라도 무시하고, 생성/수정/조회 전 과정에서 권한 검증을 서버 측에서 수행해야 합니다.

ISMS-P: ISMS-P 웹 취약점 분석·평가 방법 상세가이드의 ‘권한 종류 및 프로세스 흐름을 파악하고 통제를 우회 및 악용 가능성 점검’, ‘중간 단계를 생략하거나 우회할 수 없도록 플로우 제어 로직 추가’, ‘세션 기반 접근 통제/토큰 기반 접근 통제’ 요구사항 위반 소지가 있습니다.

4. [HIGH] 사용자 입력 **OpenAI API Key**를 서버로 전달하는 비즈니스 시크릿 오용

HIGH | Logic Bypass |

mini_project5\book\frontend\src\app\userpage\create\page.js

화면에서 OpenAI API Key를 사용자가 입력하고, 이를 그대로 /api/generateCover로 전송합니다. 이 구조는 사용자별 키를 서버가 신뢰하는 프록시로 취급하게 만들어 키 유출/재사용/오용 위험을 높이며, 서버가 인증과 무관하게 외부 API 호출을 대행하면 비용 남용 및 정책 우회가 발생할 수 있습니다. 특히 키가 서버 로그, 프록시, 예외 메시지에 노출될 경우 개인정보/계정 보안 이슈로 이어질 수 있습니다.

공격 시나리오: 공격자는 다른 사용자의 키를 입력하거나, 프록시/브라우저 확장으로 전송되는 키를 수집할 수 있습니다. 백엔드가 이 값을 그대로 OpenAI 호출에 사용하면 키 기반 과금이 무단 발생하고, 로그 노출 시 자격 증명이 탈취될 수 있습니다.

취약 코드:

```
const [userApiKey, setUserApiKey] = useState("");
...
body: JSON.stringify({
  apiKey: userApiKey,
  model: "dall-e-3",
  prompt,
}),
```

수정 코드:

```
const response = await fetch("/api/generateCover", {
  method: "POST",
  headers: { "Content-Type": "application/json", "Authorization": `Beare
  body: JSON.stringify({ model: "dall-e-3", prompt }) },
});

// 서버: 환경변수 기반 자체 키 사용
const openaiKey = process.env.OPENAI_API_KEY;
```

수정 가이드: 외부 API Key는 사용자가 입력하여 서버로 전달하는 방식보다 서버 환경 변수로 관리하는 것이 안전합니다. 사용자가 별도 키를 사용해야 하는 요구사항이 있더라도, 전송 최소화·마스킹·로그 차단·사용자별 quota/인증 연동을 서버에서 강제해야 합니다.

ISMS-P: ISMS-P 3.1.1 개인정보 수집·이용 취지 및 2.2.6 보안 위반 시 조치 관점에서, 민감한 인증정보를 적절한 통제 없이 수집·전달·처리하는 구조는 보안 위반 발생 시 책임 소재와 통제 절차를 불명확하게 만들 수 있습니다.

5. [HIGH] 클라이언트 제어 **userId**를 이용한 작품 조회/수정/삭제로 인한 IDOR 가능성

HIGH | IDOR | mini_project5\book\frontend\src\app\userpage\view\page.js

프론트엔드가 `/api/book/list/my?userId=...` 쿼리와

`formData.append("userId", user.userId)`에 의해 사용자 식별자를 클라이언트로컬스토리지 값으로 직접 전송합니다. 또한 삭제/수정 요청은 작품 ID만 경로로 전달하며, 요청 자체에 소유권을 증명하는 서버 검증 정보가 보이지 않습니다. 백엔드가 이 값을 신뢰할 경우 공격자가 `localStorage`의 `userId`를 조작하거나 다른 `bookId`를 대입해 타인의 작품을 조회/수정/삭제하는 IDOR가 성립할 수 있습니다.

공격 시나리오: 공격자가 브라우저 `localStorage`의 `user.userId`를 임의 값으로 바꾸거나,

네트워크 요청의 `userId/bookId`를 다른 사용자의 값으로 변조합니다. 서버가 인증된 세션 대신 요청 파라미터만 기준으로 권한을 판단하면 타 사용자 작품 목록을 열람하거나 임의 작품을 수정/삭제할 수 있습니다.

취약 코드:

```
const parsed = JSON.parse(user);
const userId = parsed.userId;
const response = await fetch(`/api/book/list/my?userId=${userId}`, { method: 'GET', ...
...
formData.append("userId", user.userId);
const response = await fetch(`/api/book/update/${editingWork.id}`, { method: 'PUT', ...
const response = await fetch(`/api/book/delete/${idToDelete}`, { method: 'DELETE', ...
```

수정 코드:

```
// 서버 세션/토큰 기준 사용자 식별
const response = await fetch(`/api/book/list/my`, {
  method: "GET",
  headers: { "Authorization": `Bearer ${accessToken}` }
});

// 삭제/수정 시에도 서버가 인증 주체와 리소스 소유권을 검증
const response = await fetch(`/api/book/delete/${idToDelete}`, {
  method: "DELETE",
  headers: { "Authorization": `Bearer ${accessToken}` }
});

const response = await fetch(`/api/book/update/${editingWork.id}`, {
  method: "PUT",
  headers: { "Authorization": `Bearer ${accessToken}` },
  ... (2줄 생략)
```

수정 가이드: 사용자 식별자를 클라이언트 입력값으로 신뢰하지 말고, 서버가 인증 세션/JWT의 주체(claim)로부터 사용자와 리소스 소유자를 결정하도록 변경해야 합니다. 또한 목록 조회는 `userId` 파라미터를 제거하고, 수정/삭제 API에서는 요청자와 대상 리소스의 소유권 일치 여부를 서버에서 강제 검증해야 합니다. 이는 ISMS-P의 세션/인증 관리 및 비인가 접근 방지 요구사항과도 일치합니다.

ISMS-P: ISMS-P 웹 취약점 분석·평가 항목의 세션/인증 관리 취지에 위배됩니다. 참조 1의 '16. 불충분한 세션 관리'는 안전한 토큰/세션을 사용하고 클라이언트 임의 값이 아닌 적절한 인증 주체로 접근을 통제할 것을 요구합니다. 본 코드는 `userId`를 로컬스토리지와 요청 파라미터로 직접 사용해 비인가 접근 가능성을 높이며, 제어시스템 지침(C-14)의 '세션 통신 구간에서의 타임아웃/보호'와 동일한 보호 원칙(인가 주체 확인 후 접근 허용)을 충족하지 못합니다.

6. [HIGH] 비밀번호 변경 API에서 세션 사용자 검증이 없어 타 계정 비밀번호 변경 가능

/user/change-pw 엔드포인트는 요청 본문의 `userId`를 그대로 사용해 비밀번호 변경을 수행하며, 현재 로그인 세션의 사용자와 동일한지 확인하지 않습니다. 따라서 서비스 계층이 별도 검증을 하지 않는 경우, 공격자가 임의의 `userId`를 넣어 다른 사용자의 비밀번호 변경을 시도할 수 있는 Broken Access Control(IDOR) 결함입니다. ISMS-P 관점에서도 인증 후 민감정보 변경 시점의 접근통제 미흡으로 볼 수 있으며, 비밀번호 변경은 계정 통제의 핵심이므로 영향도가 높습니다.

공격 시나리오: 공격자는 로그인한 자신의 세션이 있더라도, 다른 사용자의 `userId`를 포함한 `change-pw` 요청을 전송합니다. 서비스 계층이 요청된 `userId`만 신뢰하면, 공격자는 타인의 현재 비밀번호를 알거나 유추 가능한 경우 해당 계정의 비밀번호를 자신이 아는 값으로 변경해 계정 탈취를 수행할 수 있습니다.

취약 코드:

```
@PostMapping("/change-pw")
public ResponseEntity<?> changePassword(
    @Valid @RequestBody ChangePasswordRequest request,
    BindingResult bindingResult
) {
    ...
    userService.changePassword(
        request.getUserId(),
        request.getCurrentPassword(),
        request.getNewPassword()
    );
    ...
}
```

수정 코드:

```
@PostMapping("/change-pw")
public ResponseEntity<?> changePassword(
    @Valid @RequestBody ChangePasswordRequest request,
    BindingResult bindingResult,
    HttpSession session
) {
    if (bindingResult.hasErrors()) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body(Map.of("status", "error", "message", bindingResult.get...
    }

    Object sessionUserId = session.getAttribute("user");
    if (!(sessionUserId instanceof Long) || !sessionUserId.equals(request...
        return ResponseEntity.status(HttpStatus.FORBIDDEN)
            .body(Map.of("status", "error", "message", "권한이 없습니다."
    ... (9줄 생략)
```

수정 가이드: 비밀번호 변경은 반드시 서버가 신뢰하는 인증 컨텍스트(세션/토큰)와 연결해야 합니다. 요청 본문의 userId를 신뢰하지 말고, 세션의 사용자 식별자와 일치할 때만 변경을 허용하세요. 가능하면 userId를 요청에서 제거하고 서버가 세션 값만 사용하도록 변경하는 것이 안전합니다.

ISMS-P: ISMS-P 2.7 암호화 적용 및 계정정보 보호 관점에서, 비밀번호 변경과 같은 민감한 인증정보 처리 시 적절한 접근통제와 인증된 주체 확인이 필요합니다. 또한 계정 관리 통제(U-02 비밀번호 관리정책, U-03 계정 잠금 등)와 연계된 안전한 계정 변경 절차가 요구되며, 인증된 사용자만 자신의 비밀번호를 변경할 수 있도록 통제해야 합니다.

7. [HIGH] 비인증 사용자도 비밀번호 재설정이 가능한 권한 우회

HIGH | Broken Access Control |
mini_project5\book\src\main\java\com\aiivle0102\book\service\impl\UserService

resetPassword(String email, String name, String phone)는 호출자에 대한 인증/인가 검증 없이, 단지 이메일·이름·전화번호 일치 여부만으로 대상 계정의 비밀번호를 즉시 변경합니다. 공격자가 대상의 식별정보를 알고 있거나 추측할 수 있으면 임의로 임시 비밀번호를 발급받아 계정을 탈취할 수 있어 Broken Access Control에 해당합니다. ISMS-P 관점에서는 민감한 인증정보 변경 시 적절한 접근통제와 권한검증이 없으므로 취약합니다.

공격 시나리오: 공격자가 피해자의 이메일, 이름, 전화번호를 확보한 뒤 비밀번호 재설정 API를 호출하면 서버는 즉시 임시 비밀번호를 반환합니다. 공격자는 반환된 임시 비밀번호로 로그인하여 계정에 접근할 수 있습니다.

취약 코드:

```
UserInfo user = userInfoRepository
    .findByEmailAndNameAndPhone(email, name, phone)
    .orElseThrow(() -> new IllegalArgumentException("일치하는 사용자 정보

String tempPassword = UUID.randomUUID().toString().substring(0, 8);
user.setPassword(tempPassword);
userInfoRepository.save(user);
return tempPassword;
```

수정 코드:

```
@Transactional
public String resetPassword(String email, String name, String phone, Authen
    if (auth == null || !auth.isAuthenticated()) {
        throw new AccessDeniedException("인증이 필요합니다.");
    }
    UserInfo requester = userInfoRepository.findByEmail(auth.getName())
        .orElseThrow(() -> new AccessDeniedException("사용자 확인 실패"));
    if (!requester.getEmail().equalsIgnoreCase(email)) {
        throw new AccessDeniedException("본인 계정만 재설정할 수 있습니다.");
```

```

    }
    // 추가적으로 본인확인 (MFA/이메일 인증 링크/OTP) 후에만 변경
    UserInfo user = userInfoRepository.findByEmailAndNameAndPhone(email, name, phone)
        .orElseThrow(() -> new IllegalArgumentException("일치하는 사용자 정보 없음"));
    String tempPassword = UUID.randomUUID().toString().substring(0, 8);
    user.setPassword(passwordEncoder.encode(tempPassword));
    ... (3줄 생략)

```

수정 가이드: 비밀번호 재설정은 단순 개인정보 매칭이 아니라, 요청자 본인성 검증과 세션 기반 인가를 반드시 거치도록 변경해야 합니다. 가능하면 이메일 링크/OTP/MFA 같은 별도 검증 수단을 추가하고, 비밀번호는 평문 저장이 아니라 강한 단방향 해시로 저장해야 합니다.

ISMS-P: ISMS-P 2.7 암호화 적용 및 접근통제 관점 위반. 인증정보 변경 기능에서 명시적 인가 없이 민감정보를 변경할 수 있어, 제로트러스트의 '항상 검증(verify explicitly)' 원칙에 반합니다.

8. [HIGH] 평문 비밀번호 저장 및 변경으로 인한 계정정보 보호 실패

HIGH | Hardcoded Secrets |

mini_project5\book\src\main\java\com\aiivle0102\book\service\impl\UserService

signUp(), resetPassword(), changePassword() 모두 비밀번호를 평문으로 저장하거나 평문 비교를 수행합니다. 이는 계정정보가 유출될 경우 즉시 재사용 가능한 인증정보 노출로 이어지며, ISMS-P의 안전한 비밀번호 암호화 요구에 위배됩니다. 참조된 지침의 U-13/D-08 취지에 따라 비밀번호는 SHA-2 계열 이상의 안전한 단방향 암호화 또는 전용 비밀번호 해시 알고리즘으로 보호해야 합니다.

공격 시나리오: DB가 유출되면 모든 사용자의 비밀번호가 평문으로 노출되어 즉시 계정 탈취가 가능합니다. 또한 내부자나 로그/덤프 노출만으로도 인증정보가 재사용될 수 있습니다.

취약 코드:

```

user.setPassword(request.getPassword());
...
user.setPassword(tempPassword);
...
if (!user.getPassword().equals(currentPassword)) {
    throw new IllegalArgumentException("현재 비밀번호가 일치하지 않습니다.");
}
...
user.setPassword(newPassword);

```

수정 코드:

```

@Transactional
public UserInfo signUp(UserSignUpRequest request) {

```

```

        UserInfo user = new UserInfo();
        user.setEmail(request.getEmail().trim().toLowerCase());
        user.setPassword(passwordEncoder.encode(request.getPassword()));
        user.setName(request.getName().trim());
        user.setPhone(request.getPhone() != null ? request.getPhone().trim() :
        return userInfoRepository.save(user);
    }

    @Transactional
    public void changePassword(Long userId, String currentPassword, String newPassword) {
        UserInfo user = userInfoRepository.findById(userId)
            .orElseThrow(() -> new IllegalArgumentException("사용자를 찾을 수 없습니다"));
        if (!passwordEncoder.matches(currentPassword, user.getPassword())) {
            // ... (5줄 생략)
        }
    }

```

수정 가이드: 비밀번호는 반드시 BCrypt/Argon2 등 검증된 전용 해시를 사용해 저장하고, 비교 시에도 평문 비교가 아닌 `matches()` 방식으로 검증해야 합니다. 임시 비밀번호도 생성 즉시 해시하여 저장해야 하며, 가능하면 일회성 토큰 기반 재설정으로 전환하는 것이 안전합니다.

ISMS-P: ISMS-P 참조 U-13, D-08 및 2.7 암호화 적용 위반. 안전하지 않은 비밀번호 저장 은 계정정보 보호 실패로 직결되며, 비밀번호 암호화 정책 부재 사례에 해당합니다.

9. [MEDIUM] ID 찾기/비밀번호 찾기에서 계정 존재 여부가 노출되어 사용자 열거 가능

MEDIUM | Logic Bypass |

mini_project5\book\src\main\java\com\aiivle0102\book\controller\AuthController

`/user/find-id`와 `/user/find-pw`는 입력한 이름/전화번호/이메일 조합에 따라 성공과 실패를 명확히 구분해 응답합니다. `find-id`는 매칭 실패 시 404를 반환하고, 성공 시 이메일을 그대로 돌려주며, `find-pw`는 성공 시 임시 비밀번호 발급 여부를 응답으로 노출합니다. 이는 계정 존재 여부를 반복적으로 확인할 수 있게 해 사용자 열거를 가능하게 하므로, 계정 공격의 전단계로 악용될 수 있습니다.

공격 시나리오: 공격자는 이름/전화번호 조합 또는 이메일/이름/전화번호 조합을 대량 대입해 특정 사용자의 계정 존재 여부를 판별할 수 있습니다. `find-pw`가 임시 비밀번호를 응답에 포함하면, 통신 경로를 볼 수 있는 환경에서는 민감 인증정보 유출 위험이 커지고, 계정 탈취 시도가 쉬워집니다.

취약 코드:

```

@PostMapping("/find-id")
public ResponseEntity<?> findId(@RequestBody Map<String, String> request) {
    // ...
    if (user == null) {
        return ResponseEntity.status(404).body(Map.of("message", "일치하는 계정이 없습니다"));
    }
}

```

```

        return ResponseEntity.ok(Map.of("email", user.getEmail()));
    }

    @PostMapping("/find-pw")
    public ResponseEntity<?> findPassword(@RequestBody Map<String, String> request) {
        ...
        String tempPassword = userService.resetPassword(email, name, phone);
        data.put("tempPassword", tempPassword);
        ...
        ... (1줄 생략)
    }

```

수정 코드:

```

    @PostMapping("/find-id")
    public ResponseEntity<?> findId(@RequestBody Map<String, String> request) {
        userService.findByNameAndPhone(request.get("name"), request.get("phone"));
        return ResponseEntity.ok(Map.of("message", "요청이 처리되었습니다."));
    }

    @PostMapping("/find-pw")
    public ResponseEntity<?> findPassword(@RequestBody Map<String, String> request) {
        userService.resetPassword(request.get("email"), request.get("name"), request.get("phone"));
        return ResponseEntity.ok(Map.of("message", "요청이 처리되었습니다."));
    }

```

수정 가이드: 계정 존재 여부를 추측할 수 있는 응답 차이를 제거하고, 임시 비밀번호를 API 응답으로 내려주지 마세요. 본인확인은 별도 인증 채널(이메일/문자 인증 링크, 토큰 기반 재설정 페이지)로 처리하고, 동일한 성공 메시지로 통일해 사용자 열거를 방지해야 합니다.

ISMS-P: ISMS-P 2.7 암호화 적용에서 비밀번호는 안전한 방식으로 전달·처리되어야 하며, 안내서 결함사례처럼 비밀번호 찾기/변경 등 개인정보 전송 구간의 암호화 및 노출 통제가 중요합니다. 또한 U-13(안전한 비밀번호 암호화 알고리즘 사용) 취지에 따라 계정정보 보호를 강화해야 하며, 계정 식별 가능성 최소화는 계정 관리 보안의 기본 요구사항에 해당합니다.

10. [MEDIUM] 로그인 인증 시 비밀번호를 평문 비교하는 취약한 인증 로직

MEDIUM | Logic Bypass |

mini_project5\book\src\main\java\com\aiivle0102\book\service\impl>LoginService

LoginServiceImpl.getUser()는 사용자가 입력한 이메일과 비밀번호를 그대로 findByEmailAndPassword(email, pw)에 전달하여 계정을 조회합니다. 이 구조는 비밀번호가 DB에 평문 또는 비교 가능한 형태로 저장/조회되고 있음을 강하게 시사하며, 안전한 단방향 해시 검증(예: bcrypt/argon2, 최소 SHA-256 이상) 없이 인증이 수행될 가능성이 높습니다. ISMS-P 참조 기준상 U-13 / D-08 / 2.7 암호화 적용에 위배될 소지가 있으며, 계정정보 노출 시 크리덴셜 스테핑 및 비밀번호 유추 공격에 취약합니다.

다.

공격 시나리오: 공격자는 가입/탈퇴한 계정의 평문 비밀번호 재사용 또는 유출된 계정정보를 이용해 동일한 이메일/비밀번호 조합을 대입하여 로그인할 수 있습니다. 또한 DB 유출 시 비밀번호가 안전하게 해시되어 있지 않다면 다수 계정의 인증정보가 즉시 노출될 수 있습니다.

취약 코드:

```
String email = user.getEmail();
String pw = user.getPassword();

return userInfoRepository
    .findByEmailAndPassword(email, pw)
    .orElseThrow(() -> new RuntimeException("로그인 실패"));
```

수정 코드:

```
String email = user.getEmail();
String rawPassword = user.getPassword();

UserInfo savedUser = userInfoRepository.findByEmail(email)
    .orElseThrow(() -> new RuntimeException("로그인 실패"));

if (!passwordEncoder.matches(rawPassword, savedUser.getPasswordHash())) {
    throw new RuntimeException("로그인 실패");
}
return savedUser;
```

수정 가이드: 비밀번호를 DB에서 직접 비교하지 말고, 사용자 식별자(이메일)로만 조회한 뒤 안전한 단방향 해시 알고리즘(bcrypt/argon2 등)으로 저장된 해시와 `matches()` 방식으로 검증해야 합니다. 기존 평문/취약 해시가 저장되어 있다면 전 계정 비밀번호 재설정이 필요합니다.

ISMS-P: ISMS-P 2.7 암호화 적용 위반 소지. 참조 U-13(안전한 비밀번호 암호화 알고리즘 사용), D-08(SHA-256 이상의 안전한 해시 알고리즘 사용), 결함사례 3(안전하지 않은 MD5 사용) 및 사례 5(인증정보 평문 저장)와 유사한 위험이 있습니다.

11. [MEDIUM] 회원가입 시 이메일 중복 검사와 저장 사이의 경쟁 상태 가능성

MEDIUM | Race Condition |

mini_project5\book\src\main\java\com\aivle0102\book\service\impl\UserService

`signUp()`은 `findByEmail()`로 중복을 확인한 뒤 `save()`를 수행하는데, 별도의 유니크 제약이나 원자적 잠금이 코드상 보이지 않습니다. 동시에 동일 이메일로 요청이 들어오면 두 트랜잭션이 모두 중복 없음으로 판단하고 저장될 수 있어 계정 중복 생성이

~~발생할 수 있습니다. 이는 TOCTOU 기반의 Race Condition입니다.~~

공격 시나리오: 공격자가 동일 이메일로 회원가입 요청을 동시에 여러 개 전송하면, 중복 확인을 각각 통과한 뒤 두 건 이상이 저장될 수 있습니다. 이후 이메일 단일성에 의존하는 로그인, 비밀번호 재설정, 계정 식별 로직이 혼선에 빠질 수 있습니다.

취약 코드:

```
userInfoRepository.findByEmail(request.getEmail())
    .ifPresent(user -> {
        throw new IllegalStateException("이미 가입된 이메일입니다.");
    });

UserInfo user = new UserInfo();
user.setEmail(request.getEmail().trim().toLowerCase());
...
return userInfoRepository.save(user);
```

수정 코드:

```
@Transactional
public UserInfo signUp(UserSignUpRequest request) {
    String email = request.getEmail().trim().toLowerCase();
    if (userInfoRepository.existsByEmail(email)) {
        throw new IllegalStateException("이미 가입된 이메일입니다.");
    }
    UserInfo user = new UserInfo();
    user.setEmail(email);
    ...
    return userInfoRepository.save(user);
}

// DB 레벨에도 UNIQUE 제약 추가
// ALTER TABLE user_info ADD CONSTRAINT uk_user_email UNIQUE (email);
```

수정 가이드: 애플리케이션 레벨 중복 체크만 믿지 말고, DB 유니크 제약으로 원자성을 보장해야 합니다. 또한 중복 예외 발생 시 적절히 처리해 동시 가입 경쟁 조건을 차단해야 합니다.

ISMS-P: ISMS-P 관점에서 데이터 무결성 보호 미흡. 동시성 제어 부재로 동일 식별자 중복 생성이 가능해 계정 관리의 무결성과 통제성이 약화됩니다.

12. [INFO] 서버 비밀번호 재설정 로직 미확인으로 인한 취약점 판단 불가

INFO | Logic Bypass |

mini_project5\book\frontend\src\app\find\password\page.js

~~현재 분석 대상은 /api/user/find-pw API를 호출하는 프론트엔드 페이지입니다. 화면에서 임시 비밀번호를 출력하지만, 이 값이 어떤 방식으로 생성·검증·만료되는지, 사용자 식별 정보 검증이 어떻게 수행되는지, 레이트리밋이나 1회성 토큰이 적용되는지는 서버 코드가 없어 확인할 수 없습니다. 따라서 Broken Access Control, Input Logic Bypass, Race Condition, Hardcoded Secrets 중 실제 취약점을 이 파일만으로 확인할 수 없습니다.~~

13. [INFO] 추가 분석 불가: 로그인 프론트엔드 단독으로는 실제 비즈니스 로직 취약점 확정 어려움

INFO | N/A | mini_project5\book\frontend\src\app\login\page.js

탐색 과정에서 /api/user/login, localStorage 기반 사용자 저장, userId 활용 여부를 확인하려 했으나 도구상 파일 경로 조회가 실패하여 후속 데이터 흐름을 확인하지 못했습니다. 현재 코드만 보면 로그인 성공 후 서버 응답의 userId와 email을 localStorage에 저장하지만, 이것만으로는 곧바로 권한우회 취약점이 성립한다고 단정할 수 없습니다. 후속 페이지나 API가 이 값을 권한 판단에 사용해야만 IDOR/권한우회가 실제로 발생합니다.

14. [INFO] 현재 코드만으로는 실제 접근통제 우회 취약점 확정 불가

INFO | Broken Access Control |
mini_project5\book\src\main\java\com\aiivle0102\book\controller\UserControlle

이 컨트롤러는 세션 속성 user 존재 여부만으로 로그인 화면 접근 여부를 제어합니다. 다만 이는 '로그인 폼 접근'의 UX 분기일 뿐이며, 실제 비인가 접근이 가능한 보호 리소스가 존재하는지와 그 리소스가 동일한 세션 검사에만 의존하는지는 확인되지 않았습니다. 따라서 실익 있는 Broken Access Control로 단정하기 어렵습니다.

15. [INFO] 현재 제공 코드만으로는 비밀번호 암호화 취약점 확정 불가

INFO | Logic Bypass |
mini_project5\book\src\main\java\com\aiivle0102\book\domain\UserInfo.java

password 필드가 단순 String으로 정의되어 있으나, 엔티티 정의만으로는 서비스 계층에서 안전한 해시(SHA-2 이상, bcrypt/argon2 등)를 적용하는지 판단할 수 없습니다. ISMS-P 참조(U-13, D-08, 암호화 적용) 관점에서 점검 대상은 유효하지만, 현재 코드만으로는 익스플로잇 가능한 결함이 입증되지 않습니다.

본 리포트는 자동화 소스코드 취약점 점검을 통해 산출된 보안 감사 문서입니다.
본 결과는 참고용이며 배포 전 보안 부서의 추가 검토가 필요할 수 있습니다.